

Universidad Nacional  
**Federico Villarreal**

Facultad de Ingeniería Industrial y de Sistemas

Sesión N° 06:  
**Patrones de Arquitectura I**

Asignatura: **ARQUITECTURA DE SOFTWARE**

**Dra. Karin Rojas Romero**

Semestre Académico 2026-2



## Conflicto cognitivo

¿Cómo segmentar el software para que los módulos puedan desarrollarse por separado, admitiendo portabilidad, modificabilidad y reutilización?  
Y ¿Cómo mantener la Funcionalidad de la interfaz de usuario separada de la funcionalidad de la aplicación?

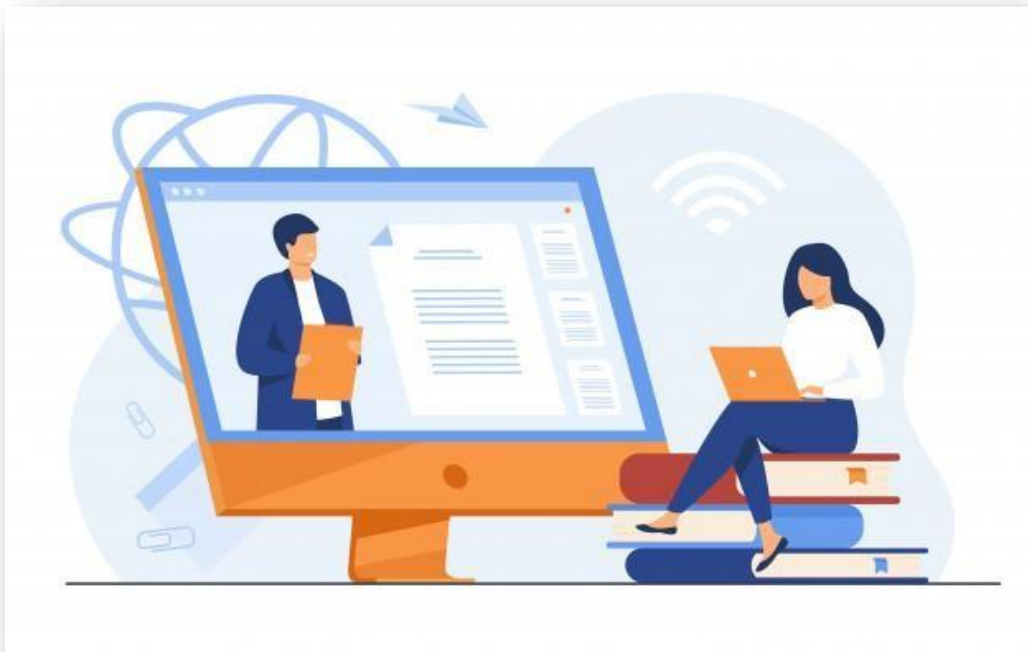


# Logro

Al término de la sesión, el estudiante desarrolla una práctica sobre patrones de arquitectura, aplicando los patrones Capas y Modelo-Vista-Controlador, de forma correcta y respetando el tiempo establecido.



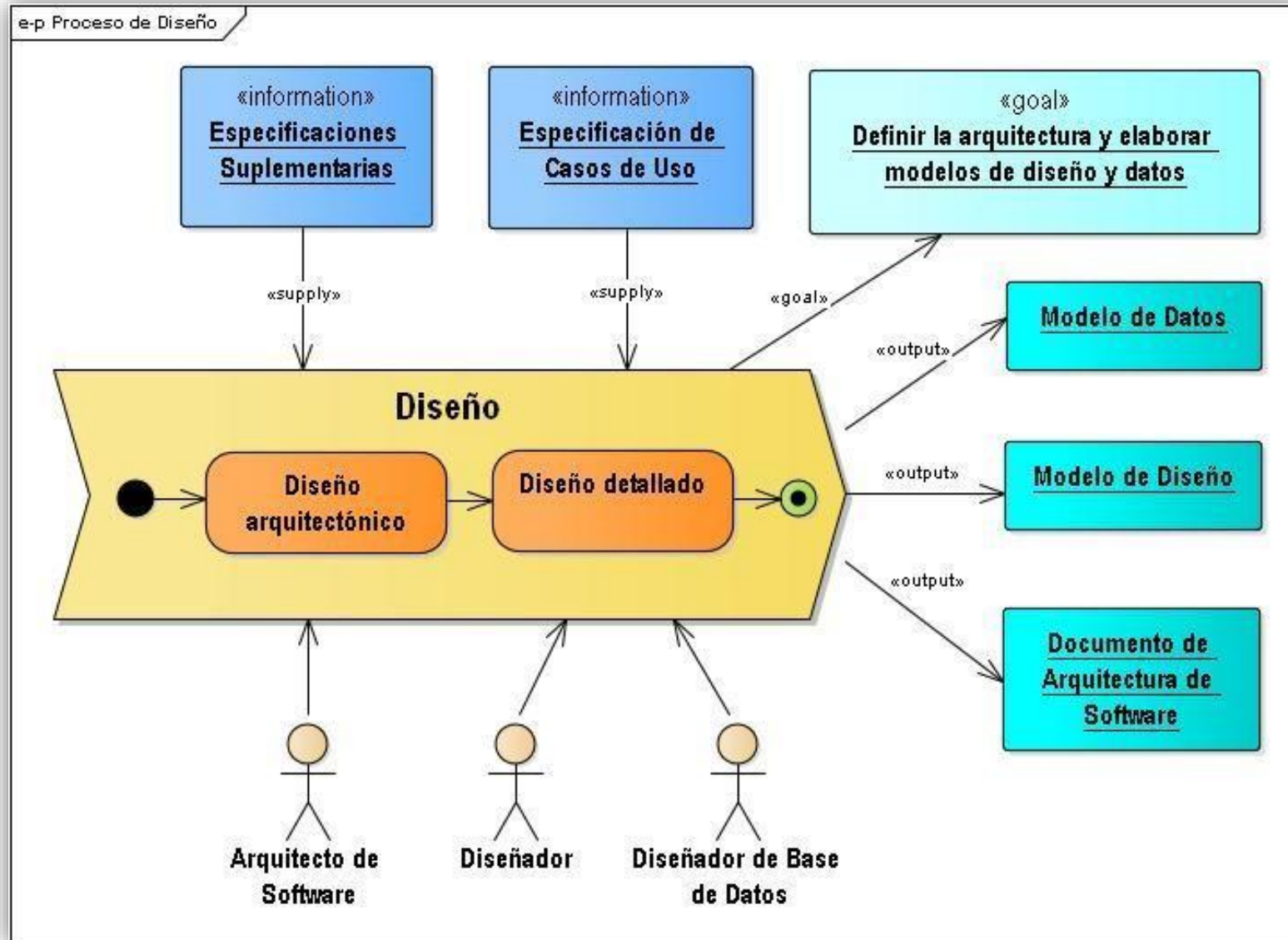
# PATRONES DE ARQUITECTURA – PARTE 1



- El Proceso de Diseño.
- El Patrón Capas.
- El Patrón Modelo-Vista-Controlador.
- Proyecto MarketSoft y diagramas de paquetes UML.



# El Proceso de Diseño





# Patrones arquitectónicos

Un patrón arquitectónico establece una relación entre:

- **Un contexto.** Una situación recurrente y común en el mundo que da lugar a un problema.
- **Un problema.** El problema, convenientemente generalizado, que surge en el contexto dado. La descripción del patrón esboza el problema y sus variantes, y describe cualquier fuerza complementaria u opuesta. La descripción del problema suele incluir los atributos de calidad que deben cumplirse.
- **Una solución.** Una solución arquitectónica satisfactoria del problema, convenientemente abstraída. La solución describe las estructuras arquitectónicas que resuelven el problema, incluida la forma de equilibrar las numerosas fuerzas en juego. La solución describirá las responsabilidades y las relaciones estáticas entre los elementos (utilizando una estructura de módulos), o describirá el comportamiento en tiempo de ejecución y la interacción entre los elementos (estableciendo una estructura de componentes y conectores o de asignación).



# Patrones arquitectónicos

La solución de un patrón se determina y describe mediante:

- Un conjunto de tipos de elementos (por ejemplo, repositorios de datos, procesos y objetos).
- Un conjunto de mecanismos de interacción o conectores (por ejemplo, llamadas a métodos, eventos o bus de mensajes).
- Una disposición topológica de los componentes.
- Un conjunto de restricciones semánticas que abarcan la topología, el comportamiento de los elementos y los mecanismos de interacción.

La descripción de la solución también debe dejar claro qué atributos de calidad proporcionan las configuraciones estáticas y de ejecución de los elementos.

Esta forma {contexto, problema, solución} constituye una plantilla para documentar un patrón.

Los sistemas complejos presentan múltiples patrones a la vez. Un sistema basado en la web puede emplear un patrón arquitectónico cliente-servidor de tres niveles, pero dentro de este patrón también puede utilizar la replicación, proxies, cachés, cortafuegos, MVC, etc., cada uno de los cuales puede emplear más patrones y tácticas. Y todas estas partes del patrón cliente-servidor probablemente emplean capas para estructurar internamente sus módulos de software.





# Patrones de módulo: Patrón Capas

- **Contexto:** Todos los sistemas complejos experimentan la necesidad de desarrollar y hacer evolucionar partes del sistema de forma independiente. Por esta razón, los desarrolladores del sistema necesitan una separación de intereses clara y bien documentada, de modo que los módulos del sistema puedan desarrollarse y mantenerse de forma independiente.
- **Problema:** El software tiene que estar segmentado de tal manera que los módulos puedan desarrollarse y evolucionar por separado con poca interacción entre las partes, apoyando la portabilidad, la modificabilidad y la reutilización.
- **Solución:** Para lograr esta separación de preocupaciones, el patrón de capas divide el software en unidades llamadas capas. Cada capa es una agrupación de módulos que ofrece un conjunto cohesionado de servicios. Hay restricciones en la relación permitida entre las capas: las relaciones deben ser unidireccionales. Las capas particionan completamente un conjunto de software, y cada partición se expone a través de una interfaz pública. Las capas se crean para interactuar según una relación de orden estricta.





# Patrones de módulo: Patrón Capas

Por supuesto, nada de esto es gratis. Alguien debe diseñar y construir las capas, lo que a menudo puede agregar costos iniciales y complejidad a un sistema. Además, si las capas no están diseñadas correctamente, en realidad pueden interferir, al no proporcionar las abstracciones de nivel inferior que los programadores de niveles superiores necesitan. Y las capas siempre agregan una penalización de rendimiento a un sistema. Si se realiza una llamada a una función en la capa superior, es posible que tenga que atravesar muchas capas inferiores antes de ser ejecutada por el hardware. Cada una de estas capas agrega algo de sobrecarga propia, como mínimo en forma de cambio de contexto.



# Solución del Patrón Capas

**Visión general:** El patrón de capas define las capas (agrupaciones de módulos que ofrecen un conjunto cohesivo de servicios) y una relación unidireccional de uso permitido entre las capas. El patrón suele mostrarse gráficamente apilando cajas que representan capas unas sobre otras.

**Elementos:** Capa, un tipo de módulo. La descripción de una capa debe definir qué módulos contiene la capa y una caracterización del conjunto cohesionado de servicios que proporciona la capa.

**Relaciones:** Permitido para usar, que es una especialización de una relación dependiente más genérica. El diseño debe definir cuáles son las reglas de uso de la capa (por ejemplo, "una capa puede usar cualquier capa inferior" o "una capa puede usar solo la capa inmediatamente debajo de ella") y cualquier excepción permitida.

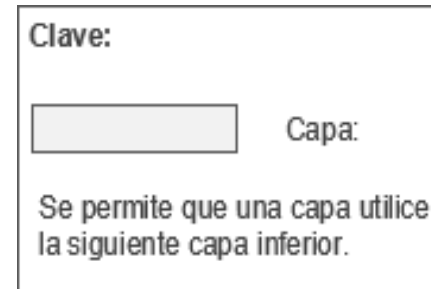
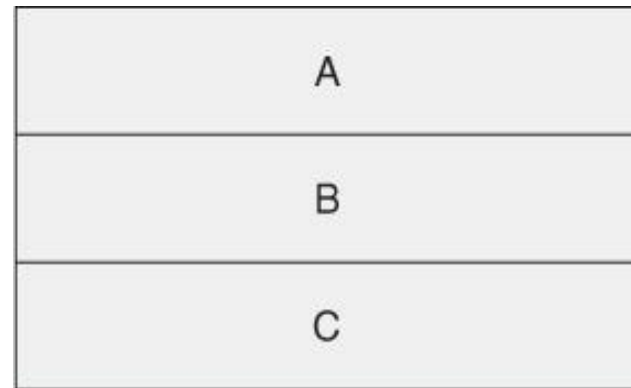
**Restricciones:** Cada pieza de software se asigna exactamente a una capa. Hay al menos dos capas (pero normalmente hay tres o más). Las relaciones de uso permitido no deben ser circulares (es decir, una capa inferior no puede utilizar una capa superior).

**Puntos débiles:** La adición de capas agrega un costo inicial y complejidad a un sistema. Las capas contribuyen a una penalización del rendimiento.



# Esquema del Patrón Capas

Las capas se dibujan casi siempre como una pila de cajas. La relación de uso permitido se denota por adyacencia geométrica y se lee de arriba hacia abajo, como en la figura.

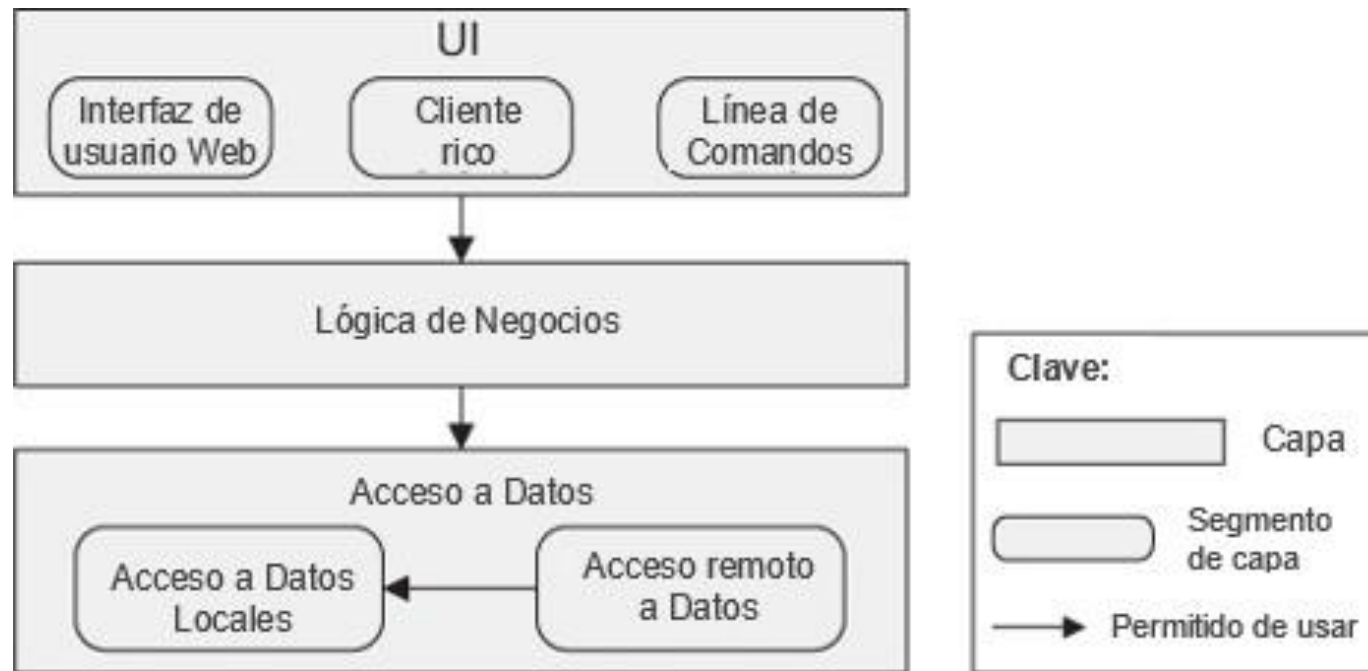


Notación de pila de cajas para diseños en capas



# Esquema del Patrón Capas

A veces, las capas se dividen en segmentos que denotan una descomposición más fina de los módulos.

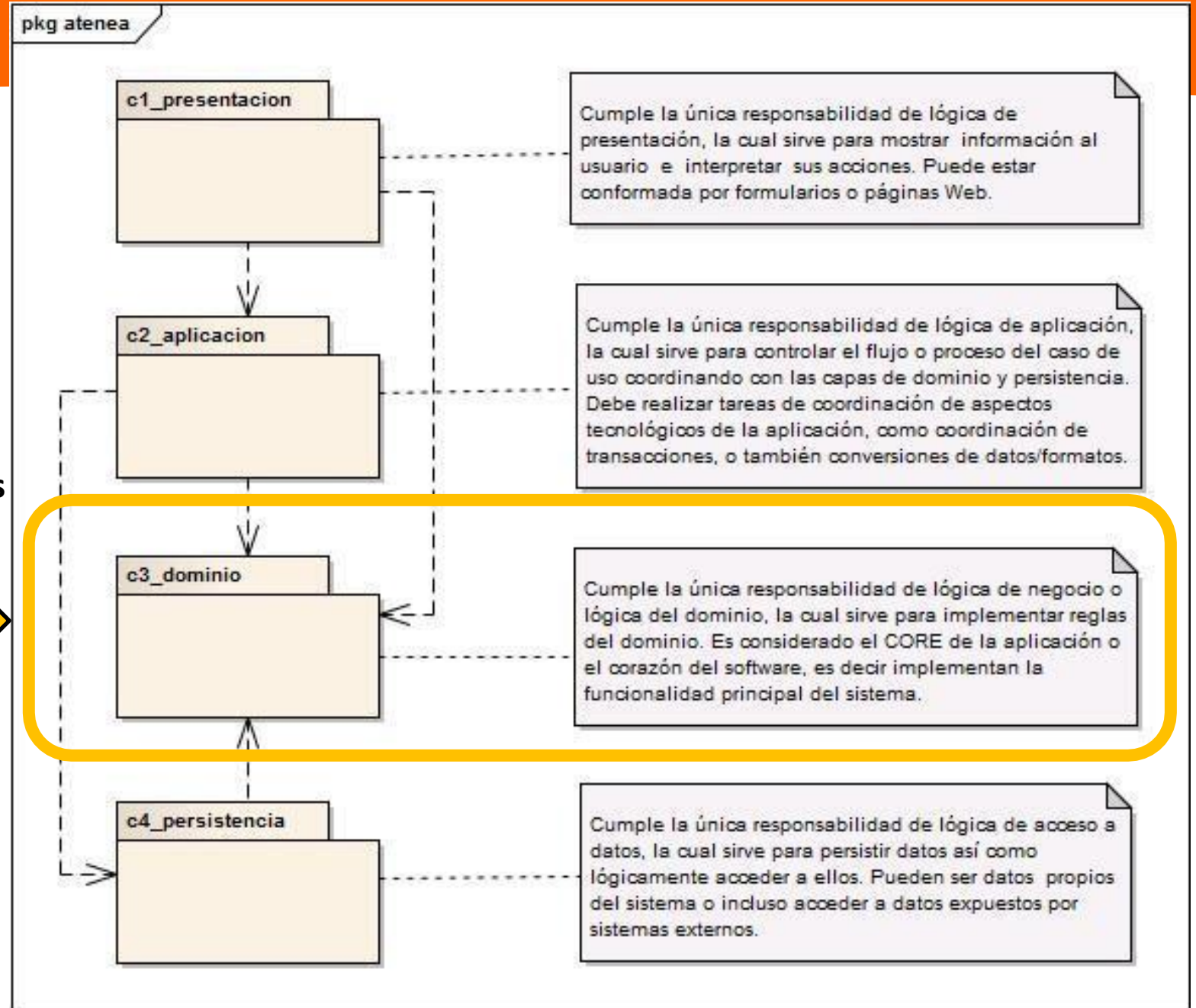


Diseño en capas con capas segmentadas



# Diseño del Patrón N- Capas DDD con diagramas de paquetes de UML

La capa más  
importante





# Patrones de componentes y conectores:

## Patrón Modelo-Vista-Controlador (MVC)

- **Contexto:** El software de interfaz de usuario suele ser la parte que más se modifica de una aplicación interactiva. Por este motivo, es importante mantener las modificaciones del software de interfaz de usuario separadas del resto del sistema. Los usuarios suelen querer ver los datos desde diferentes perspectivas, como un gráfico de barras o un gráfico circular. Ambas representaciones deben reflejar el estado actual de los datos
- **Problema:** ¿Cómo se puede mantener la funcionalidad de la interfaz de usuario separada de la funcionalidad de la aplicación y, sin embargo, responder a la entrada del usuario o a los cambios en los datos de la aplicación subyacente? ¿Y cómo se pueden crear, mantener y coordinar múltiples vistas de la interfaz de usuario cuando cambian los datos de la aplicación subyacente?
- **Solución:** El patrón modelo-vista-controlador (MVC) separa la funcionalidad de la aplicación en tres tipos de componentes: un modelo, que contiene los datos de la aplicación; una vista, que muestra una parte de los datos subyacentes e interactúa con el usuario; un controlador, que media entre el modelo y la vista y gestiona las notificaciones de los cambios de estado.



# Patrón Modelo-Vista-Controlador (MVC)

El MVC no es apropiado para todas las situaciones. El diseño e implementación de tres tipos distintos de componentes, junto con sus diversas formas de interacción, puede ser costoso, y este coste puede no tener sentido para interfaces de usuario relativamente sencillas. Además, la correspondencia entre las abstracciones de MVC y los conjuntos de herramientas de interfaz de usuario comerciales no es perfecta. La vista y el controlador separan la entrada y la salida, pero estas funciones suelen combinarse en widgets individuales. Esto puede dar lugar a un desajuste conceptual entre la arquitectura y el kit de herramientas de interfaz de usuario.





# Solución del Patrón Modelo-Vista-Controlador

**Visión General (MVC):** El patrón MVC divide la funcionalidad del sistema en tres componentes: un modelo, una vista y un controlador que media entre el modelo y la vista.

**Elementos:** El *modelo* es una representación de los datos o el estado de la aplicación y contiene (o proporciona una interfaz a) la lógica de la aplicación. La *vista* es un componente de la interfaz de usuario que produce una representación del modelo para el usuario o permite alguna forma de entrada del usuario, o ambas cosas. El *controlador* gestiona la interacción entre el modelo y la vista, traduciendo las acciones del usuario en cambios en el modelo o cambios en la vista.

**Relaciones:** La relación de notificaciones conecta instancias de modelo, vista y controlador, notificando a los elementos de los cambios de estado relevantes.

**Restricciones:** Debe haber al menos una instancia de cada modelo, vista y controlador. El componente del modelo no debe interactuar directamente con el controlador.

**Puntos débiles:** Es posible que la complejidad no valga la pena para interfaces de usuario simples. Es posible que las abstracciones de modelo, vista y controlador no sean adecuadas para algunos kits de herramientas de interfaz de usuario.



# Patrón Modelo-Vista-Controlador (MVC)

De hecho, puede haber muchas vistas y muchos controladores asociados a un modelo. Por ejemplo, un conjunto de datos empresariales puede representarse como columnas de números en una hoja de cálculo, como un gráfico de dispersión o como un gráfico circular. Cada uno de ellos es una vista separada, y esta vista puede ser actualizada dinámicamente a medida que el modelo cambia (por ejemplo, mostrando transacciones en vivo en un sistema de procesamiento de transacciones). Un modelo puede actualizarse mediante diferentes controladores; por ejemplo, un mapa puede ampliarse y desplazarse mediante movimientos del ratón, clics del teclado o comandos de voz; cada una de estas diferentes formas de entrada debe ser gestionada por un controlador.

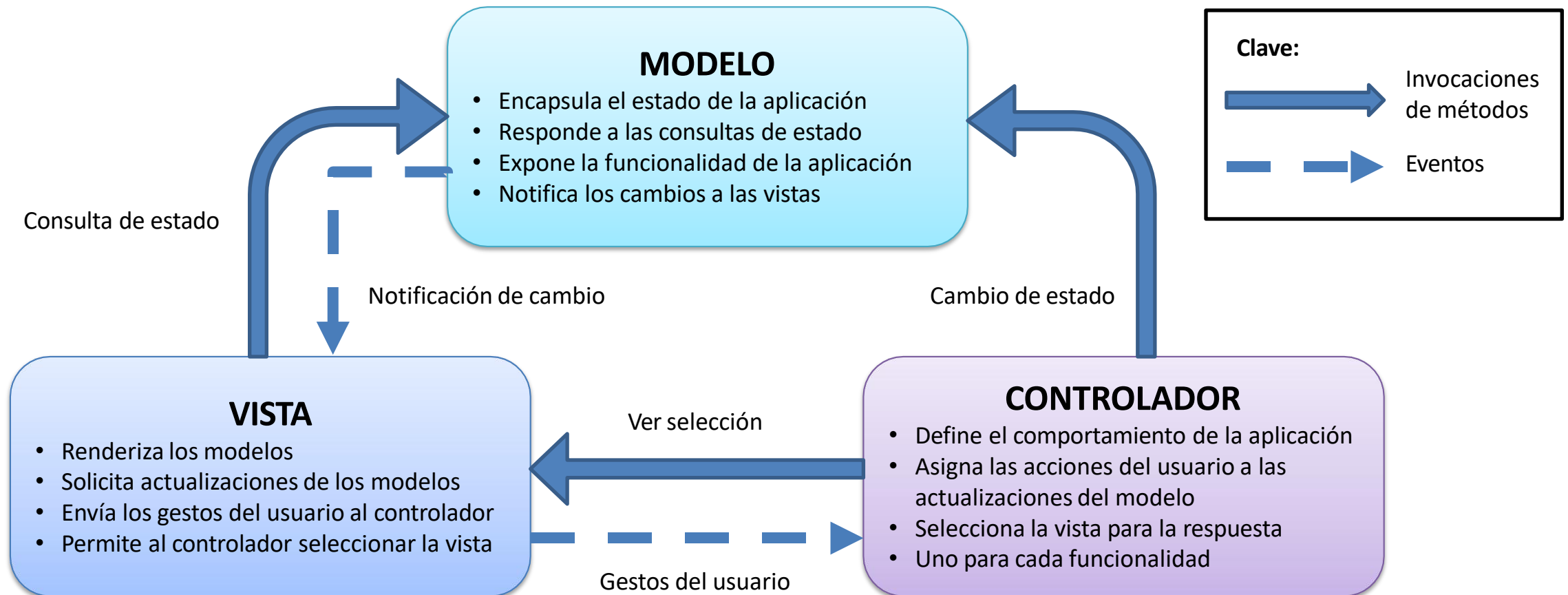
Los componentes MVC están conectados entre sí a través de algún tipo de notificación, como eventos o devoluciones de llamada. Estas notificaciones contienen actualizaciones de estado. Un cambio en el modelo debe comunicarse a las vistas para que puedan actualizarse. Un evento externo, como una entrada de usuario, debe comunicarse al controlador, que a su vez puede actualizar la vista y / o el modelo.

Como estos componentes están poco acoplados, es fácil desarrollarlos y probarlos en paralelo, y los cambios en uno de ellos tienen un impacto mínimo en los demás.



# Esquema del Patrón Modelo-Vista-Controlador

Las relaciones entre los componentes de MVC se muestran en la figura.



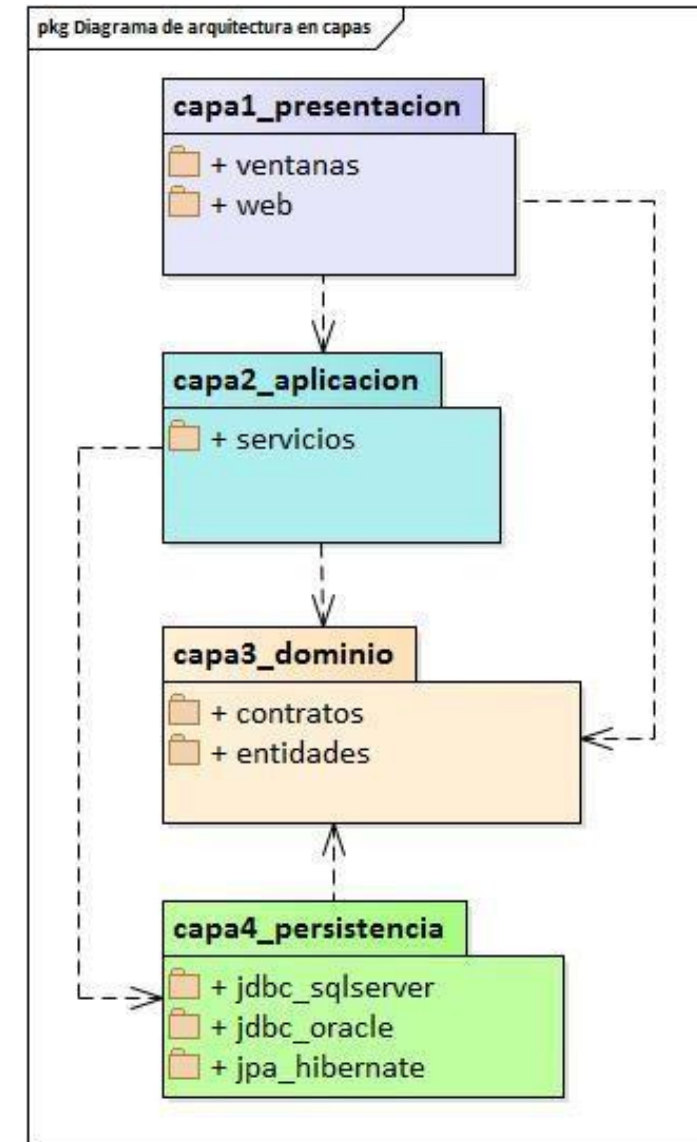


# Proyecto MarketSoft



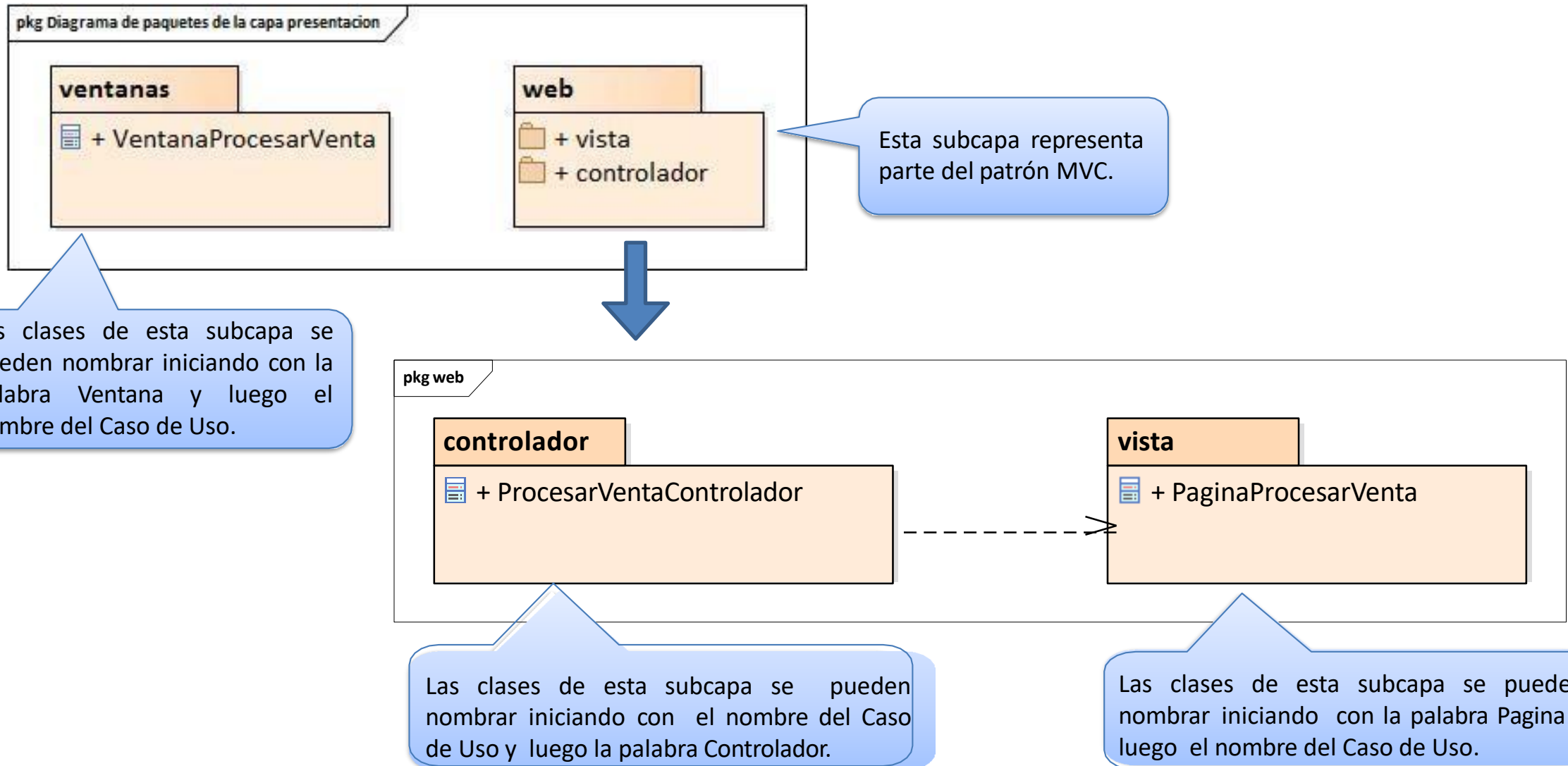


# Diagrama de paquetes UML de la Arquitectura N-Capas DDD



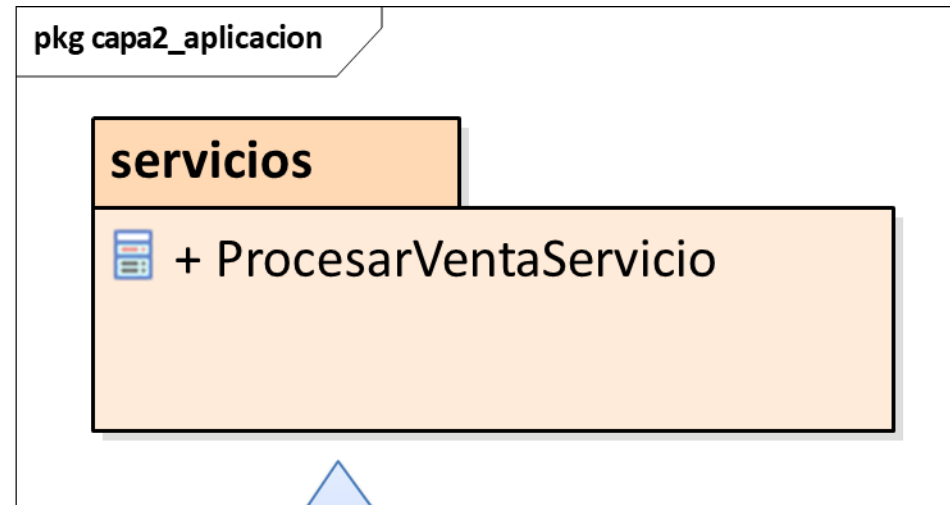


# Diagrama de paquetes UML de la capa





# Diagrama de paquetes UML de la capa Aplicación

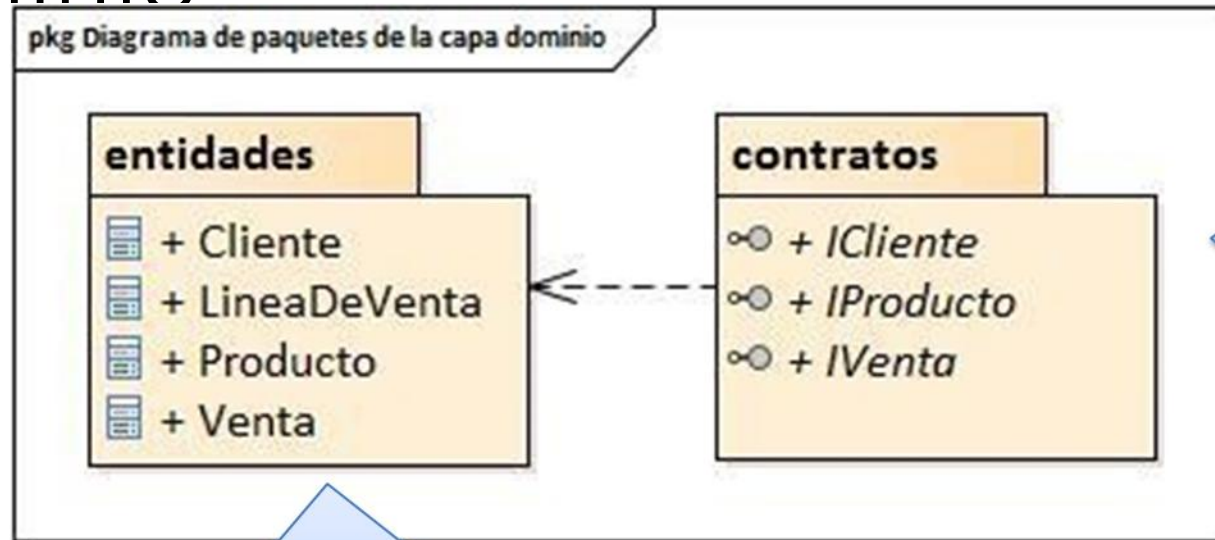


Las clases de esta subcapa se pueden nombrar iniciando con el nombre del Caso de Uso y luego la palabra Servicio.





# Diagrama de paquetes UML de la capa Dominio



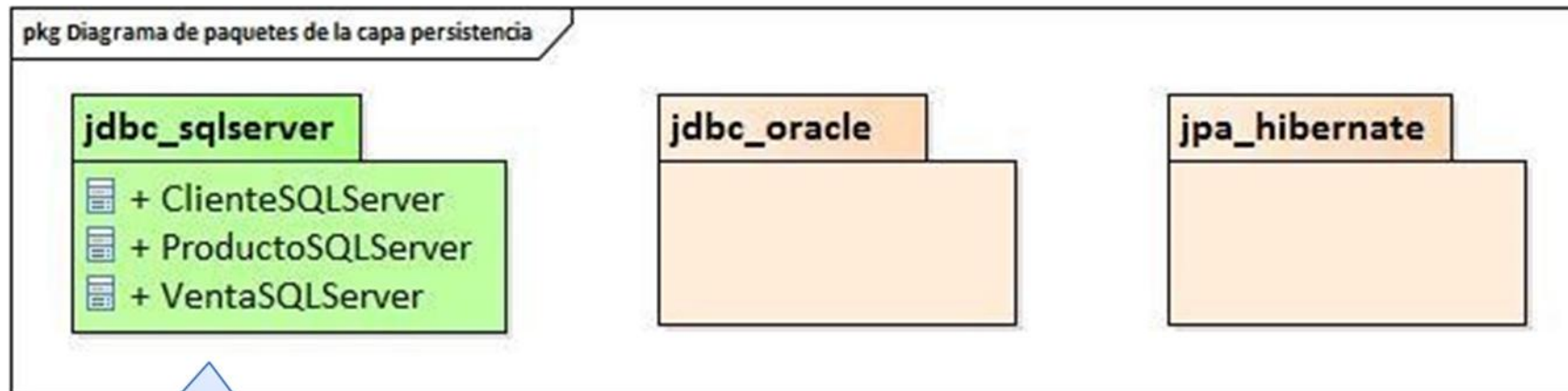
Esta subcapa no contiene clases, tendrá sólo interfaces que declaran métodos de acceso a los datos. Las interfaces de esta capa serán implementadas por las clases de la capa de persistencia.

Las clases de esta subcapa se deben identificar y nombrar en función a los conceptos que representan información para el dominio o negocio y que son descritos en los casos de uso del sistema.

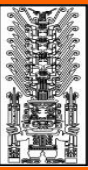


# Diagrama de paquetes UML de la capa Persistencia

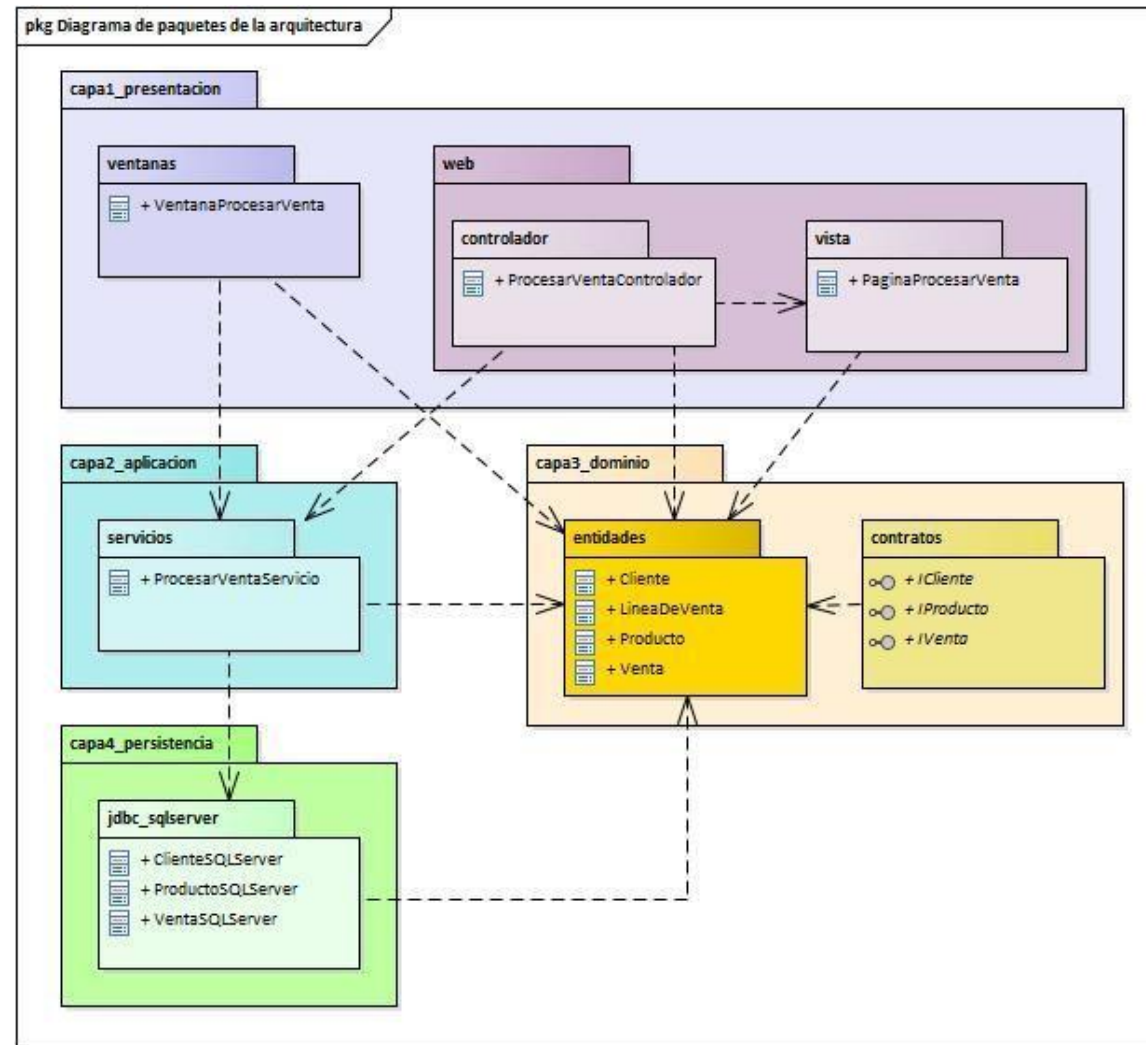
Los nombres de las subcapas corresponden a la tecnología usada para el acceso a los datos persistentes.



Las clases de esta subcapa se pueden nombrar iniciando con el nombre de las clases de Entidad y luego una palabra que represente la tecnología de acceso a datos.



# Diagrama completo de la Arquitectura N-Capas DDD





# Trabajo en equipo



Desarrolla las actividades propuestas durante la sesión con tu equipo de trabajo.



# Preguntas de metacognición

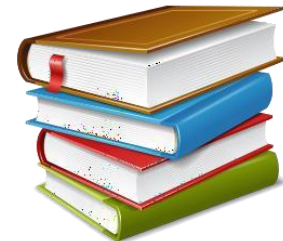


1. ¿Qué aprendí?
2. ¿Cómo lo aprendí?
3. ¿Para qué me sirve lo que aprendí?
4. ¿Cómo hemos distribuido las tareas?
5. ¿Qué dificultades tuve al realizar las tareas?
6. ¿Cómo me sentí en el proceso?



# Bibliografía

- ❖ Len Bass, Paul Clements, Rick Kazman. (2013). Software Architecture in Practice. Tercera Ed. Addison Wesley.
  - ✓ Capitulo 13 - Tácticas y patrones arquitectónicos.
  
- ❖ Craig Larman, (2003) “UML Y PATRONES una Introducción al Análisis y Diseño Orientado a Objetos y al Proceso Unificado”, 2da edición.
  - ✓ Capitulo 30: Diseño de la Arquitectura lógica con patrones.







**Mg. Manuel Narro Andrade**  
**[mnarro@unfv.edu.pe](mailto:mnarro@unfv.edu.pe)**

*Gracias*

